

Đường Rẽ Còn Xa, Trên Dưới Kiếm Tìm

Bàn qua ví dụ về “sâu” và “rộng” trong phân tích thi tin học

Xiao Hanjun
Giáo viên hướng dẫn: Chen Ying
Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tháng 1 năm 2008

Nguồn gốc: On depth and breadth in analysis for informatics competitions

IOI-China-Candidate-Team-Papers/2008/Day1/6-Xiao-Hanjun/xiao-hanjun-paper.pdf

Abstract

Bài viết là một số suy nghĩ và đúc kết của tác giả trước những thay đổi mới trong thi tin học. Trọng tâm là ý nghĩa đặc biệt của hai yêu cầu “sâu” và “rộng” trong phân tích bài toán thi tin học, cách đặt được hai yêu cầu ấy, và một vài điểm cần chú ý trong quá trình học tập thường ngày.

Từ khóa: phương pháp tư duy, phân tích sâu, nhiều góc độ, quy nạp.

1 Biển Rộng Sóng Dài: Ý Nghĩa Của “Sâu” Và “Rộng” Trong Tin Học

Người ta thường dùng “sâu” và “rộng” để tả biển cả; nhưng theo một nghĩa khác, biển trong tư duy con người còn sâu thẳm và bao la hơn bất kỳ đại dương nào. Thi tin học là một sân khấu để năng lực tư duy và phân tích được bộc lộ đầy đủ. Vậy trong sân khấu ấy, “sâu” và “rộng” có ý nghĩa đặc biệt gì? Làm thế nào để phân tích vừa sâu vừa rộng? Bài viết này cùng bạn đọc tìm lời giải cho các câu hỏi đó.

1.1 Thế nào là “sâu”?

Trong phân tích bài toán, “sâu” có mấy tầng ý nghĩa sau.

Thứ nhất là **tính phân tầng**. Khi mới bắt đầu phân tích, một bài toán thường không dễ tiếp cận: có bài có số chiều cao, có bài có điều kiện phức tạp. Quan sát theo tầng nghĩa là nắm lấy đầu mối của bài toán, xét từ những trường hợp ít chiều hơn hoặc điều kiện đơn giản hơn để rút ra một số tính chất. Sau đó ta đi từ nông đến sâu, phân tích xem các tính chất ấy biến đổi ra sao khi trở lại trường hợp nhiều chiều hoặc điều kiện phức tạp, nhằm tìm điểm đột phá.

Thứ hai là **tính liên tục**. Quá trình phân tích bài toán thường dài và gập ghềnh. Có lúc ý tưởng lóe lên, tưởng như đã nắm được điều gì; có lúc lại rơi vào bế tắc, cuối cùng không thu được gì. Nếu nghĩ đến đầu phân tích đến đó, tư duy dễ rời và khó giữ mục tiêu. Ngược lại, nếu kiên trì đi sâu từng bước theo một hướng xác định, tận dụng đầy đủ kết quả của mỗi bước phân tích, ta dễ đào ra được những điều hữu ích.

Thứ ba là chú ý tới **quan hệ giữa các yếu tố**. Triết học nói rằng sự vật có liên hệ phổ biến. Trong bài toán tin học, quan hệ giữa các yếu tố cũng là đối tượng rất đáng đào sâu. Bài toán không phải phép cộng cơ học của các yếu tố; quan hệ giữa chúng cũng tồn tại khách quan, thậm chí bản chất và sâu sắc hơn. Những quan hệ như đơn điệu, chu kỳ, v.v. nếu được nghiên cứu có ý thức, thường có thể trở thành điểm mở khóa lời giải.

1.2 Thế nào là “rộng”?

Nội hàm của “rộng” trong phân tích bài toán còn phong phú hơn.

Thứ nhất là **tầm nhìn mở**. Trong thời gian gấp gáp của phòng thi, nhiều bài rất khó, thậm chí không thể, có lời giải hoàn chỉnh. Khi đó ta cần phát huy trí tưởng tượng và dùng nhiều chiến lược để xử lý bài toán. Có thể là thuật toán ngẫu nhiên, có thể là tham lam, thậm chí có thể thiết kế thuật toán riêng cho từng dạng dữ liệu có khả năng xuất hiện. Chiến lược khác nhau thì hiệu quả thực tế của chương trình cũng rất khác nhau.

Thứ hai là **mạch nghĩ rộng**. Cùng một bài toán, nhìn từ góc độ khác nhau thường cho ra kết quả khác nhau. Nếu tổng hợp được kết quả phân tích từ nhiều góc nhìn, ta có thể hiểu bài toán tương đối toàn diện; những thông tin ẩn, khi bị thăm dò từ nhiều hướng, cũng không còn chỗ ẩn náu.

Thứ ba là **phương tiện phân tích phong phú**. Phương tiện phân tích là lưỡi dao sắc của tư duy, giúp nắm lấy bản chất bài toán. Nhưng cũng như mỗi loại vũ khí đều có sở trường và sở đoản, các phương tiện phân tích cũng có đặc điểm riêng. Chẳng hạn, suy diễn toán học chính xác nhưng trừu tượng; tính chất hình học trực quan nhưng không dễ lượng hóa. Nếu nắm được nhiều phương tiện phân tích, ta giống như người luyện võ thông thạo đủ loại binh khí: trong chuyện trò ung dung, bài toán đã tan biến.

2 Mưu Định Rồi Mới Động: Phân Tích Sâu Bản Chất Bài Toán

Dưới đây, qua hai bài toán ví dụ, ta bàn về cách đạt được chiều sâu trong phân tích.

2.1 Ví dụ 1: Liệt kê số có n chữ số

Mô tả bài toán. Đếm số lượng các số có n chữ số, không chứa chữ số 0, và chia hết cho 9.¹

Dữ liệu.

$$1 \leq n \leq 500.$$

Phân tích ban đầu. Đề bài yêu cầu thống kê, nên rất dễ nghĩ đến thuật toán liệt kê.

Thuật toán 1: tìm trong các số có n chữ số. Liệt kê mọi số có n chữ số, rồi kiểm tra xem có chứa chữ số 0 hay không và có chia hết cho 9 hay không. Độ phức tạp thời gian là

$$\mathcal{O}(10^n).$$

Khai thác điều kiện đặc biệt để phân tích sâu hơn. Điều đáng chú ý là bài toán yêu cầu đếm các số đồng thời “không chứa chữ số 0” và “chia hết cho 9”. Đó chính là các điều kiện đặc biệt của bài toán. Từ hai góc nhìn này, ta có hai thuật toán khác.

Thuật toán 2: tìm trong các bội số có n chữ số của 9. Khai thác điều kiện chia hết cho 9, ta có thể trực tiếp liệt kê số có n chữ số ấy là bội thứ bao nhiêu của 9, rồi kiểm tra từng bội trong phạm vi. Hiệu quả ít nhất tốt hơn thuật toán đầu khoảng 9 lần. Độ phức tạp thời gian vẫn là

$$\mathcal{O}(10^n).$$

¹Bài toán do tác giả tự đặt.

Thuật toán 3: tìm trong các số có n chữ số không chứa 0. Nếu xem một số có n chữ số là một chỉnh hợp lặp từ 0 đến 9, thì những số không chứa chữ số 0 chính là các chỉnh hợp lặp từ 1 đến 9. Ta chỉ cần kiểm tra các số ấy có chia hết cho 9 hay không. Độ phức tạp thời gian là

$$\mathcal{O}(9^n).$$

Tuy nhiên, các thuật toán mũ này hiển nhiên không đủ cho quy mô n đã cho.

Từ nông đến sâu, từng tầng tiến lên. Nhìn lại mấy ý tưởng vừa nêu, thuật toán 1 là cách trực tiếp đơn giản nhất; thuật toán 2 tận dụng tính chất bội số, chỉ cải thiện phần nào; thuật toán 3 đưa ra cách nhìn số có n chữ số như một chỉnh hợp lặp và tự nhiên loại bỏ sự quấy nhiễu của chữ số 0. Nhưng ta thấy thuật toán 2 và 3 mỗi thuật toán chỉ dùng một tính chất đặc biệt, chưa kết hợp cả hai. Vì vậy, dựa trên thuật toán 3 và kết hợp tính chất đặc biệt của bội số của 9, ta có thuật toán 4.

Thuật toán 4: dùng tính chất đặc biệt của bội số của 9. Một số chia hết cho 9 khi tổng các chữ số của nó chia hết cho 9. Khi $n - 1$ chữ số đầu đã được cố định, vì các số $1, 2, \dots, 9$ có phần dư khác nhau khi chia cho 9, chữ số cuối cùng thực ra được xác định duy nhất. Đến đây, bài toán trở thành: có bao nhiêu chỉnh hợp lặp độ dài $n - 1$ từ 1 đến 9? Đáp án rõ ràng là

$$9^{n-1}.$$

Độ phức tạp thời gian là $\mathcal{O}(\log n)$, nếu dùng lũy thừa nhanh và tạm xem phép toán số lớn là $\mathcal{O}(1)$.

Đến đây, bài toán đã được giải trọn vẹn.

Quan hệ giữa đối tượng liệt kê và hiệu quả. Như vừa thấy, liệt kê cũng có cách kém hiệu quả và cách hiệu quả. Vậy nguyên nhân tạo ra khác biệt đó là gì?

Ký hiệu	Ý nghĩa của tập hợp
A	mọi số có n chữ số
B	các bội số có n chữ số của 9
C	các số có n chữ số không chứa 0
$D = B \cap C$	các số cần đếm

Hình 2.1. Góc nhìn tập hợp: D là phần giao của hai tập con B và C nằm trong không gian lớn A .

Từ góc nhìn tập hợp, thuật toán liệt kê nói chung là tìm một tập chứa mục tiêu cuối cùng để làm đối tượng liệt kê, chẳng hạn A , B , hoặc C trong hình trên; sau đó kiểm tra từng phần tử được liệt kê và loại phần thừa để thu được mục tiêu cuối cùng. Hiệu quả liệt kê vì thế gắn chặt với việc chọn đối tượng liệt kê. Tập tương ứng càng nhiều phần tử thì hiệu quả càng thấp; ngược lại, tập càng nhỏ thì hiệu quả càng cao.

Trong bốn thuật toán trên, thuật toán 1 gần như thao tác trực tiếp theo lời đề, tương ứng với tập A lớn nhất. Thuật toán 2 và 3 lần lượt nắm một mặt tính chất của bài toán nhưng chưa toàn diện, nên độ phức tạp chỉ cải thiện phần nào; các tập tương ứng B và C có nhỏ hơn. Thuật toán 4 kết hợp thuật toán 2 và 3, sáng tạo dựng lên một tương ứng một-một giữa bài toán gốc và một bài toán đếm khác dễ hơn, từ đó tính trực tiếp kích thước của tập D .

Chọn đối tượng liệt kê thích hợp như thế nào? Tầm quan trọng của đối tượng liệt kê đã được chứng minh đầy đủ. Nhưng làm sao chọn được đối tượng thích hợp? Từ góc nhìn của Hình 2.1 có thể rút ra vài gợi ý. Ban đầu, sau khi hiểu đề, ta mới tiếp xúc với hình thái nguyên thủy nhất của bài toán, như phần A trong hình; đối tượng liệt kê còn thô. Khi chú ý đến những tính chất đặc biệt của bài toán, đào sâu và

khai thác chúng, ta có khả năng đi từ ngoài vào trong và thu được đối tượng liệt kê tốt hơn, như B và C . Khi tiếp tục chú ý tới khác biệt và liên hệ giữa nhiều tính chất, ta có thể nắm bắt tổng hợp bản chất bài toán và thật sự chọn được đối tượng liệt kê phù hợp.

Quá trình ấy thực ra chính là quá trình phân tích bài toán ngày càng sâu. Ta phân tích càng sâu, càng gần bản chất bài toán, thì càng có khả năng tìm được thuật toán đơn giản. Trong phân tích, sự chú ý tới tính chất đặc biệt và tới quan hệ giữa nhiều tính chất giúp ta nắm chính xác hơn nội hàm và ngoại diên của bài toán, thậm chí tạo ra đột phá thuật toán.

2.2 Ví dụ 2: Sokoban

Mô tả bài toán. Bé Z là một cậu bé thông minh và ham chơi. Một ngày nọ, khi đang tìm thú vui trên mạng, một trò chơi kinh điển nhưng khó lọt vào tầm mắt cậu: Sokoban, hay đẩy hộp.²

Trò chơi Sokoban diễn ra trên một bản đồ gồm tường và ô trống. Mục tiêu là đẩy tất cả các hộp đến vị trí chỉ định. Người chơi điều khiển một nhân vật; mỗi thao tác cho nhân vật thử di chuyển theo một trong bốn hướng lên, xuống, trái, phải. Nếu ô kế tiếp là ô trống, chỉ vị trí nhân vật thay đổi. Nếu ô kế tiếp là hộp và ô phía sau hộp theo cùng hướng là ô trống, nhân vật và hộp cùng di chuyển một ô. Trong các trường hợp còn lại, thao tác không có hiệu lực.

Nhìn qua mới biết trò Sokoban này có quá nhiều hộp. Bé Z hoa mắt chóng mặt, nên quyết định chỉ chơi phiên bản có một hộp. Vốn theo đuổi sự hoàn hảo, cậu luôn muốn hoàn thành trò chơi bằng số thao tác ít nhất. Bạn có thể giúp cậu không?

Dữ liệu vào. Dòng đầu là n, m, k , lần lượt là số hàng, số cột của bản đồ và số ván chơi trên bản đồ này.

Tiếp theo là n dòng, mỗi dòng m ký tự mô tả bản đồ. Tường được ký hiệu bằng #, ô trống bằng ..

Tiếp theo là k dòng, mỗi dòng gồm 6 số nguyên, lần lượt là hàng và cột của nhân vật, của hộp và của đích. Dữ liệu bảo đảm nhân vật không thể đi ra ngoài bản đồ; nhân vật và hộp không trùng nhau và không nằm trên tường.

Dữ liệu ra. Với mỗi ván chơi, in một dòng là số thao tác ít nhất cần để hoàn thành. Nếu không thể hoàn thành, in No Answer!.

Quy mô.

$$3 \leq n, m \leq 30, \quad 1 \leq k \leq 1000.$$

Phân tích ban đầu. Vì cần số thao tác ít nhất, bài toán liên quan đến địa hình, vị trí hộp và nhiều yếu tố khác. Khi chưa tìm được mô hình thích hợp, tìm kiếm chắc chắn là một thử nghiệm có ích.

Tìm kiếm vét cạn. Dùng tìm kiếm theo chiều rộng. Mỗi bước thử di chuyển theo bốn hướng. Cắt tỉa cơ bản là tránh lặp trạng thái; một trạng thái có thể được biểu diễn bằng tọa độ của nhân vật và hộp. Nếu gặp trạng thái đã tìm kiếm trước đó thì không cần tìm tiếp.

Độ phức tạp thời gian của tìm kiếm này có thể đo bằng số trạng thái. Trong trường hợp xấu nhất, tọa độ nhân vật và hộp đều có thể đi qua mọi ô trống, do đó:

$$\text{độ phức tạp thời gian} = \mathcal{O}(kn^2m^2).$$

²Bài toán do tác giả tự đặt.

Tách hai loại di chuyển. So với quy mô bài toán, độ phức tạp của thuật toán thứ nhất vẫn chưa thỏa mãn. Quá trình phân tích ví dụ 1 gợi ý rằng nếu phát hiện được tính chất riêng của bài toán, ta có thể có đột phá mới.

Nhìn lại bài toán một lần nữa, ta thấy chuyển động của nhân vật thực ra có quy luật nhất định: hoặc là đẩy hộp, hoặc là thực hiện một chuỗi di chuyển chuẩn bị cho lần đẩy hộp tiếp theo. Nhưng chương trình tìm kiếm của ta không buộc nhân vật lúc nào cũng tuân thủ hai quy tắc ấy. Thường xuyên hơn, khi không đẩy hộp, nhân vật chỉ chạy lòng vòng trên bản đồ. Nếu ép nhân vật đi theo đường ngắn nhất tới mục tiêu kế tiếp, dường như có thể tránh tìm kiếm không cần thiết.

Tìm kiếm hai tầng. Theo ý tưởng trên, ta chuyển đổi trạng thái tìm kiếm từ bản thân thao tác sang đường đi của hộp. Vì khi hộp di chuyển thì nhân vật nhất định đứng ở một trong bốn phía của hộp, số trạng thái khác nhau chỉ còn $\mathcal{O}(nm)$. Đường đi của nhân vật khi không đẩy hộp có thể được xử lý bằng một BFS phụ. Trong trường hợp thông thường, BFS phụ này kết thúc rất nhanh và hiệu quả chương trình tăng rõ rệt. Tuy nhiên trong tình huống cực đoan, chẳng hạn các ngõ cụt, BFS này vẫn có thể duyệt toàn bộ bản đồ.

Ngoài ra cần chú ý rằng vì ta không mở rộng trạng thái theo đúng thứ tự số thao tác từ ít đến nhiều, đôi khi cùng một trạng thái phải được tìm kiếm lặp. Nhưng điều kiện để lặp tìm khá ngặt, nên ảnh hưởng tới độ phức tạp thời gian là rất nhỏ.

$$\text{độ phức tạp thời gian} = \mathcal{O}(kn^2m^2).$$

Cùng một bản đồ. Phân tích tới đây, bài toán tưởng như khó tiến thêm. Nhưng trong các thuật toán vừa nêu, ta chưa hề dùng một tính chất: cần trả lời k ván chơi khác nhau, nhưng tất cả đều dùng cùng một bản đồ. Một bản đồ bất biến lẽ nào lại không chứa thông tin bất biến? Nếu có thể tiền xử lý bằng cách nào đó để đào lấy thông tin của bản đồ, có lẽ tình thế sẽ sáng lên.

Từ tìm kiếm sang đường đi ngắn nhất. Dưới gợi ý ấy, khi soi lại tìm kiếm hai tầng, ta chợt thấy: phần do BFS phụ xử lý chẳng phải đúng là phần tách khỏi ván chơi cụ thể và chỉ liên quan tới bản đồ hay sao? Chỉ cần liệt kê trước vị trí hộp, thực hiện các tìm kiếm và lưu trữ cần thiết, trong k ván chơi sau đó sẽ không cần tìm kiếm mới.

Nhưng phân tích không nên dừng lại ở đây. Di chuyển của nhân vật khi không đẩy hộp có thể tiền xử lý, vậy di chuyển đẩy hộp cũng có thể tiền xử lý hay không? Nếu xem mỗi trạng thái trong thuật toán hai tầng là một đỉnh, xem việc chuyển từ trạng thái này sang trạng thái khác bằng một số thao tác là cạnh, và gán trọng số cạnh bằng số thao tác, bài toán gốc có thể được biểu diễn bằng mô hình đường đi ngắn nhất đơn giản. Điều cần chú ý chỉ là điểm bắt đầu và điểm kết thúc đều có thể có tới đa bốn trạng thái; thêm một đỉnh nguồn và một đỉnh đích mới là đủ để xử lý.

$$\text{độ phức tạp thời gian} = \mathcal{O}(n^2m^2 + knm),$$

nếu dùng SPFA; vì đồ thị thưa và có tính phân tầng mạnh, hiệu quả thực tế rất tốt.

Nhìn lại. Có thể nói ba thuật toán trên đều được xây dựng trên việc phân tích ngày càng sâu, và giữa chúng có liên hệ chặt chẽ. Nếu không thử tìm kiếm vét cạn, có lẽ ta không phát hiện được việc nhân vật chạy lòng vòng vô ích, từ đó tách hai loại thao tác của nhân vật. Nếu không cải tiến biểu diễn trạng thái trong tìm kiếm hai tầng, có lẽ ta khó biến bài toán tìm kiếm thành bài toán đồ thị và dùng thuật toán hoàn toàn mới để giải bài gốc.

Vì vậy, tư tưởng “phân tích sâu bản chất bài toán” còn yêu cầu ta nắm lấy những tính chất và điều kiện chưa được sử dụng, tìm khả năng kết hợp chúng với kết quả phân tích đã có, bóc tách từng lớp để từng bước lộ ra bản chất bài toán.

2.3 Kéo dài bài toán theo chiều dọc

Điều đáng nói là giải được bài không có nghĩa bài toán đã kết thúc. Thay đổi điều kiện của bài toán một cách thích hợp và suy nghĩ xem trong tình huống mới bài toán biến đổi ra sao sẽ giúp ta hiểu bài toán sâu sắc hơn.

Lấy ví dụ thứ hai. Sokoban là bài toán rất quen thuộc, lời giải tổng quát thường dùng tìm kiếm sâu dần, còn tối ưu có hiệu quả tốt là tìm kiếm hai tầng như đã nêu. Khi suy nghĩ về hiệu quả thời gian của tìm kiếm hai tầng, tác giả chú ý rằng tầng tìm kiếm thứ hai chỉ liên quan tới bản đồ, từ đó thay đổi bài toán và xử lý nó bằng đồ thị.

Ví dụ thứ nhất cũng có thể kéo dài theo chiều dọc. Trong bài, chữ số bị cấm chỉ là 0, yêu cầu chia hết cũng chỉ là chia hết cho 9. Nếu mở rộng tập chữ số bị cấm thành một tập con tùy ý của $0, 1, \dots, 9$, và mở rộng điều kiện chia thành: phần dư khi chia cho p không được thuộc $r_1, r_2, \dots, r_k$, bài toán sẽ có những biến đổi mới. Bài toán cụ thể và lời giải có thể tham khảo cuộc thi tự đặt của tác giả.

3 Nhìn Ngang Thành Dãy Núi, Nhìn Nghiêng Thành Đỉnh: Tư Duy Nhiều Góc Độ

Tiếp theo, qua hai bài toán ví dụ khác, ta bàn về cách đạt được độ rộng trong phân tích.

3.1 *Caves and Tunnels*

Mô tả bài toán. Cho một cây có n đỉnh và Q thao tác. Mỗi thao tác hoặc tăng trọng số của một đỉnh, hoặc hỏi trọng số lớn nhất trên đường đi giữa hai đỉnh. Ban đầu mọi trọng số đều bằng 0.³

Dữ liệu.

$$1 \leq n, Q \leq 100000.$$

Phân tích. Nếu thao tác diễn ra trên một dãy tuyến tính, đây là bài toán RMQ động kinh điển và có thể giải dễ dàng bằng cây đoạn. Nhưng đối tượng thao tác của bài này lại là một cây. Ta nên mở rộng thao tác trên dãy sang cây như thế nào? Mô phỏng cách giải trên dãy bằng chia để trị, hay quy về dãy rồi giải bằng công cụ tuyến tính? Có vẻ có nhiều hướng, ta có thể thử từng hướng một.

Chia để trị trên cây. Chú ý rằng khi giải bài toán trên dãy, tư tưởng cơ bản của ta là chia để trị. Khi bài toán mở rộng sang cây, chia để trị trên cây dường như cũng là một ý tưởng hợp lý.

Tuy nhiên, thử thêm một chút sẽ thấy cây không có tính tuyến tính, nên quan hệ giữa điểm chia và điểm truy vấn khó biểu diễn. Vì thế ta không thể giống như trên dãy: khi một đoạn được chứa trọn thì lấy ngay dữ liệu thống kê của đoạn ấy ra. Sau nhiều nỗ lực không có kết quả, thử nghiệm chia để trị thất bại.

³Ural 1553, Novosibirsk SU Contest, Petrozavodsk training camp, September 2007.

Quy về dãy tuyến tính. Một ý tưởng khác là quy bài toán về dãy tuyến tính. Tính chất “giữa hai điểm bất kỳ trong cây chỉ có một đường đi” thật ra cũng có liên hệ với dãy.

Ý tưởng đơn giản nhất là với mỗi đường đi được hỏi, dựng một dãy riêng cho nó. Dĩ nhiên cách này quá chậm: số đường đi quá nhiều. Nhưng nếu giảm số lượng đường đi thì sao?

Từ gợi ý đó, ta tùy ý chọn một đỉnh làm gốc. Khi đó đường đi nối hai đỉnh có thể được chia tại tổ tiên chung gần nhất của chúng thành hai đoạn để xử lý. Như vậy, mọi đường đi đều hướng về gốc. Nếu dựng một dãy cho đường đi từ mỗi đỉnh đến gốc thì quả thật có thể giải bài, nhưng chi phí thời gian và bộ nhớ khó chấp nhận. Có cách nào tốt hơn không?

Kết hợp hai ý tưởng. Hai ý tưởng vừa rồi tuy chưa giải được bài, nhưng mạch nghĩ trong đó vẫn có thể dùng. Tư duy nhiều góc độ không phải là bắn một phát rồi đối súng; tìm một cách có ý thức khả năng kết hợp các ý tưởng khác nhau có thể mở cục diện và đưa ta ra khỏi bế tắc.

Chú ý rằng ý tưởng dãy tuyến tính bị chặn lại vì số dãy cần dựng quá nhiều. Mà chia để trị chẳng phải là công cụ mạnh để giảm quy mô bài toán hay sao? Theo hướng này, mỗi lần ta tạo một dãy cho đường đi từ lá sâu nhất trong cây hiện tại đến gốc; các nhánh còn lại được xử lý đệ quy. Nói cách khác, xem một cây như được hợp bởi nhiều chuỗi, rồi chuyển bài toán trên cây thành nhiều bài toán trên chuỗi. Có thể chứng minh rằng trong trường hợp xấu nhất, mỗi truy vấn liên quan tới $\mathcal{O}(\sqrt{n})$ dãy.

$$\text{độ phức tạp thời gian} = \mathcal{O}(Q\sqrt{n}).$$

3.2 Paper

Mô tả bài toán. Bé D cần đọc n bài viết. Mỗi bài có thời gian đọc T_i và giá trị V_i . Chi phí đọc một bài bằng thời điểm từ 0 đến lúc đọc xong bài đó nhân với giá trị của nó. Bé D muốn sắp xếp thứ tự đọc sao cho tổng chi phí là nhỏ nhất.⁴

Ngoài ra, một số bài có cùng tác giả, và bé D muốn đọc các bài của cùng một tác giả theo đúng thứ tự người ấy viết ra.

$$\boxed{T_1, V_1} \mid \boxed{T_2, V_2} \Rightarrow \boxed{T_2, V_2} \mid \boxed{T_1, V_1}$$

Hình 3.1. Đổi chỗ hai bài kề nhau.

Dữ liệu.

$$1 \leq n \leq 50000.$$

Phân tích ban đầu. Bài toán yêu cầu một thứ tự đọc tối ưu để tổng chi phí nhỏ nhất, đồng thời có thêm ràng buộc: các bài của cùng tác giả phải được đọc theo thứ tự thời gian. Nếu xét trực tiếp bài toán gốc, ta khó tìm được hướng tốt; vì vậy trước hết hãy thử làm yếu điều kiện và giải bài gốc từ nông đến sâu.

Bỏ ràng buộc. Ta thử bỏ yếu tố tác giả và xét lời giải trong trường hợp đặc biệt này.

Bài toán yêu cầu một dãy tối ưu. Rõ ràng dãy tối ưu không phải một hoán vị tùy tiện; nó phải có điểm đặc biệt nào đó để trở thành tối ưu. Điểm đặc biệt ấy là: sau bất kỳ thay đổi nào, kết quả thu được cũng không tốt hơn dãy ban đầu.

Trong một dãy, thay đổi đơn giản nhất là hoán đổi hai phần tử kề nhau. Ta theo hướng đó để phân tích sâu hơn.

⁴Cuộc thi lập trình sinh viên tỉnh Quảng Đông lần thứ hai, năm 2004.

Xét hai cuốn sách kề nhau, có thời gian đọc T_1, T_2 và giá trị V_1, V_2 . Như Hình 3.1, sau khi đổi thứ tự đọc hai cuốn sách, thời điểm đọc xong cuốn thứ nhất bị trễ thêm T_2 , còn cuốn thứ hai sớm hơn T_1 . Do đó biến thiên của tổng chi phí là

$$T_2 V_1 - T_1 V_2.$$

Với dãy tối ưu, nhất định phải có

$$T_2 V_1 > T_1 V_2, \quad \text{tức là} \quad \frac{V_1}{T_1} > \frac{V_2}{T_2}.$$

Nếu mọi cặp kề nhau đều có quan hệ này, toàn bộ dãy chính là thứ tự giảm dần theo $\frac{V_i}{T_i}$. Đến đây, trường hợp đặc biệt đã được giải nhẹ nhàng.

Trường hợp chung đặc biệt nhất. Trở lại bài toán chung, ta xử lý ảnh hưởng của yếu tố tác giả tới thứ tự đọc như thế nào? Vẫn dùng tư tưởng đi từ đặc biệt tới tổng quát.

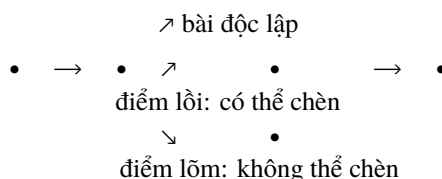
Trong mọi trường hợp chung, trường hợp chỉ có một tác giả là đặc biệt nhất, nhưng thứ tự đã cố định nên không có giá trị nghiên cứu. Tiếp theo là trường hợp chỉ có hai tác giả. Trong các trường hợp hai tác giả, trường hợp một tác giả chỉ viết một bài lại là đặc biệt nhất. Ta hãy xét xem bài độc lập ấy nên được đọc vào lúc nào.

Tương tự, bằng cách nghiên cứu biến thiên tổng chi phí sau một thay đổi bất kỳ, ta khảo sát tính chất đặc biệt mà thứ tự đọc tối ưu cần có. Vì thứ tự các bài cùng tác giả cố định, không được thay đổi, nên thay đổi phải bao gồm bài độc lập kia. Theo phân tích vừa rồi, trong dãy tối ưu, các phần tử kề nhau có thể hoán đổi phải thỏa:

$$\frac{V_i}{T_i} > \frac{V_{i+1}}{T_{i+1}}.$$

Tức là tỉ số của bài độc lập phải nhỏ hơn tỉ số phía trước và lớn hơn tỉ số phía sau. Nhưng điều đó có nghĩa gì? Chỉ nhìn từ suy diễn toán học, có thể có nhiều vị trí thỏa bất đẳng thức trên, chưa có tính chất tốt để khai thác. Dạng tỉ số của một cặp số lại khiến ta liên tưởng tới hệ số góc của đường thẳng. Ở đây, hãy thử hình học hóa bài toán và dùng một phương tiện phân tích khác.

Dùng hình để trợ giúp số. Xem mỗi bài viết là một vector (T_i, V_i) ; khi đó tỉ số $\frac{V_i}{T_i}$ chính là hệ số góc của vector ấy. Nối các vector đầu-cuối liên tiếp và biểu diễn chúng trong mặt phẳng tọa độ. Khi đó vị trí chèn bài độc lập được mô tả trực quan là một “điểm lồi”.



Hình 3.2. Dùng hình để trợ giúp số: điểm lồi cho vị trí có thể chèn, còn điểm lõm phải bị loại bằng cách gộp hai vector tạo ra nó.

Dĩ nhiên, chỉ chú ý một mặt của bài toán là trái với yêu cầu “rộng”. Hãy chú ý các vị trí không thể chèn; trong hình học, chúng biểu hiện thành “điểm lõm”. Có thể dễ dàng dùng phản chứng để chứng minh rằng điểm lõm không thể được bài mới chèn vào; nếu không, đổi nó với vector phía trước hoặc phía sau sẽ cho lời giải tốt hơn. Nói cách khác, để xuất hiện điểm lõm, ta có thể gộp hai vector tạo ra điểm lõm ấy và loại bỏ điểm lõm.

Hình nào không chứa điểm lõm? Đúng vậy, đó là bao lồi. Hệ số góc các cạnh trên bao lồi lại vừa đúng là đơn điệu giảm. Có thể cảm nhận rằng ta chỉ còn cách đáp án cuối cùng một bước.

Thuật toán cuối cùng. Xử lý riêng từng tác giả và tìm bao lỗi do các bài viết của tác giả đó tạo thành. Các điểm trên bao lỗi chia các bài của tác giả ấy thành nhiều đoạn; hệ số góc trong mỗi đoạn đơn điệu giảm. Sau đó sắp xếp tất cả các đoạn bài theo hệ số góc giảm dần, ta thu được thứ tự tối ưu cần tìm. Để chứng minh dãy này hợp lệ, vì các đoạn bài của cùng tác giả tự thân đã thỏa thứ tự hệ số góc giảm dần, nên việc sắp xếp sẽ không phá vỡ thứ tự viết bài.

$$\text{độ phức tạp thời gian} = \mathcal{O}(n \log n).$$

Nhìn lại. Quá trình giải bài này có thể nhìn lại qua bốn bước:

1. Trước hết ta bỏ ràng buộc, xuất phát từ trường hợp đặc biệt và thu được quan hệ giữa các phần tử kề nhau trong dãy tối ưu.
2. Trong trường hợp chung, ta chọn trường hợp hai tác giả đặc biệt nhất để thảo luận, từ đó rút ra tính chất toán học của vị trí chèn.
3. Sau đó ta dùng phương pháp kết hợp số và hình, khảo sát điểm lỗi và điểm lồi từ cả hai phía thuận và nghịch.
4. Cuối cùng liên tưởng tới bao lỗi và giải bài toán liền mạch.

Có thể thấy mỗi bước trong quá trình giải bài đều ít nhiều liên hệ với “sâu” và “rộng”. Tư duy nhiều góc độ thực chất là cố gắng đạt độ rộng trong phân tích, nhưng điều này không có nghĩa nhìn bài toán từ các góc độ tách rời nhau. Ta cần nhằm vào các tính chất ở những mặt khác nhau của chính bài toán, tổng hợp để nắm nội hàm và ngoại diện, phần đặc biệt và phần tổng quát của nó. Khi dùng nhiều tính chất và nhiều mô hình để giải bài, cũng không phải cộng cơ học hay áp dụng mù quáng, mà phải tổ hợp chúng hữu cơ và sáng tạo để đạt hiệu quả tốt nhất.

3.3 So sánh ngang nhiều hơn

Trong quá trình luyện tập thường ngày, cũng không nên thỏa mãn với một lời giải duy nhất. Hấp thụ nhiều nét độc đáo trong phân tích của người khác và thử dùng chúng vào lúc thích hợp có thể mở rộng mạch nghĩ. Những chỗ trước đây không nghĩ tới, sau khi tiếp nhận ý tưởng mới, có thể được xét đến nhẹ nhàng hơn.

Mặt khác, so sánh các lời giải khác nhau cũng rất quan trọng. Hiệu quả của các lời giải thường không giống nhau. Nếu phân tích được nguyên nhân tạo ra khác biệt, hiểu biết về từng lời giải cũng sâu hơn. Khi gặp một bài có nhiều lời giải, ta cũng có thể cân nhắc toàn diện hơn đặc điểm của từng cách, nhờ đó quyết định dễ dàng hơn.

4 Mài Dao Không Làm Chậm Việc Chặt Củi: Một Vài Gợi Ý Cho Việc Học Lập Trình

4.1 Rèn thói quen phân tích tốt

Trong thực hành giải bài thường ngày, ta hay dựa vào kinh nghiệm và dùng các phương pháp phân tích một cách không tự giác. Nhưng hiệu quả lúc tốt lúc xấu, khó nắm chắc. Điều đó cho thấy kinh nghiệm rời rạc không thể thay thế lý thuyết có hệ thống. Nếu trong quá trình phân tích có thể tuân theo một số quy tắc, phân tích có lý và có thứ tự, ta sẽ tránh được sự bất ổn do chỉ dựa vào kinh nghiệm; đó cũng chính là lợi ích lớn nhất của một thói quen phân tích tốt.

4.2 Nghĩ nhiều, trao đổi thường xuyên

Như đã nói ở trên, độ sâu và độ rộng của suy nghĩ phân tích quyết định chất lượng giải bài. Mặt khác, mục tiêu cuối cùng của thi tin học cũng là bồi dưỡng năng lực tư duy của thí sinh. Vì vậy, trong quá trình học và luyện tập thường ngày, việc tự giác phân tích sâu bài toán từ nhiều góc độ là rất có ích. Đồng thời cần chú ý rằng nếu chỉ làm một mình, hoặc chỉ trao đổi khép kín trong một nhóm nhỏ, ta không dễ mở rộng mạch nghĩ và hấp thụ ý tưởng mới. Ngược lại, nếu có thể tìm hiểu rộng rãi cách người khác nghĩ về cùng một bài toán, ta sẽ phân tích linh hoạt hơn.

Về con đường trao đổi thì không cần gò bó; mỗi người có thể dùng sở trường của mình. Trong thời đại thông tin phong phú và liên lạc ngày càng nhanh, ta vừa có thể lấy dưỡng chất từ sách và bài viết, vừa có thể chất lọc từ mạng; vừa có thể thảo luận trực tiếp, vừa có thể trao đổi vượt không gian trên mạng. Hình thức có thể đa dạng, nhưng mục tiêu đều là nâng cao năng lực tư duy. Nếu kiên trì, chắc chắn sẽ có tiến bộ.

4.3 Đối xử đúng với bài toán kinh điển

Bài toán kinh điển được gọi là “kinh điển” không chỉ vì thường gặp, mà còn vì trong đó chứa những phương pháp tư duy tin học cơ bản nhưng quan trọng. Trong luyện tập hằng ngày, các bài kinh điển thường chỉ được xem như tri thức “chết” để ghi nhớ; khi gặp bài tương tự thì sửa nhẹ rồi áp vào. Cách làm đó bỏ qua tác dụng rèn luyện tư duy của bài toán kinh điển.

Chẳng hạn thuật toán sắp xếp nhanh quen thuộc có thể xem là kinh điển. Nhưng nhiều bạn chỉ dừng ở việc biết dùng nó để sắp xếp, mà không chú ý tới tư tưởng cơ bản: chọn một số đặc biệt để phân hoạch dãy và chia để trị. Vì vậy, khi gặp bài tìm phần tử lớn thứ k trong dãy, không ít bạn thường sắp xếp trước rồi in ra, với độ phức tạp $\mathcal{O}(n \log n)$. Nhưng nếu hiểu tư tưởng chia để trị trong sắp xếp nhanh, ta hoàn toàn có thể thiết kế thuật toán nhanh hơn: chọn nhanh.

Tư tưởng cơ bản của chọn nhanh cũng giống sắp xếp nhanh. Trước hết, trong phạm vi đang xét, chọn một số làm mốc; nhờ đó có thể chia toàn bộ dãy thành hai hoặc ba đoạn. Sau đó, dựa vào giá trị của k , quyết định đoạn nào cần tiếp tục xử lý. Độ phức tạp thời gian của cách này là $\mathcal{O}(n)$, cụ thể hơn, nếu chọn mốc ngẫu nhiên thì độ phức tạp kỳ vọng là

$$n + \frac{n}{2} + \frac{n}{4} + \cdots + 1 = \mathcal{O}(n),$$

đạt tới cận dưới lý thuyết.

Ví dụ này minh họa sinh động ý nghĩa quan trọng của bài toán kinh điển. Nó cũng nhắc rằng trong quá trình học bài kinh điển, ta nên hiểu chúng từ bản chất tư tưởng. Hãy thật sự xem bài toán kinh điển như sự phơi bày các liên hệ khách quan giữa các đại lượng trong một mô hình cụ thể.

4.4 Đúc kết và quy nạp thích hợp

Khi đã làm nhiều bài và tích lũy được một số mạch nghĩ, ta cần chỉnh lý chúng cẩn thận và rút ra những điều quan trọng. Việc này có nhiều lợi ích. Một là củng cố trí nhớ, tiêu hóa những ý tưởng thu được để biến chúng thành một phần của mình. Hai là trong quá trình đúc kết, ta có thể so sánh các mạch nghĩ theo chiều ngang và chiều dọc; trong sự va chạm ấy, biết đâu lại có phát hiện mới.

Bài viết này cũng chính là một suy nghĩ và đúc kết tương đối có hệ thống của tác giả trong nhiều năm thi đấu. Những mô tả về các vấn đề thường gặp ở từng giai đoạn học tập trong bài đều xuất phát từ trải nghiệm của tác giả. Vì thế tác giả cảm nhận sâu sắc tầm quan trọng của việc đúc kết và quy nạp.

Tài liệu tham khảo

1. Liu Rujia. *Những gợi ý từ bài toán người khuân vác*. 2001.
2. Zhou Gelin. *Báo cáo lời giải “Query on a tree” (SPOJ 375)*. 2006.
3. Yang Zhe. *Một số nghiên cứu về lời giải QTREE*. 2007.

Lời cảm ơn

Nhân dịp hoàn thành bài viết này, tôi xin bày tỏ lòng cảm ơn chân thành tới các thầy cô, bạn học, bạn bè và gia đình đã quan tâm và giúp đỡ tôi trong nhiều năm qua.

Xin cảm ơn cô Chen Ying vì sự quan tâm và hướng dẫn.

Xin cảm ơn huấn luyện viên Liu Rujia và anh Hu Botao vì những góp ý chỉnh sửa cho bài viết.

Xin cảm ơn các thành viên đội tuyển tập huấn Yu Linyun và Chen Danqi vì sự giúp đỡ.

Xin cảm ơn toàn thể thành viên tổ tin học Trường Trung học số 1 Phúc Châu vì sự ủng hộ và khích lệ.

Tôi xin dành bài viết này cho cha mẹ kính yêu nhất của tôi.